TOPCASINOSG · FOR DEVELOPERS AND AI INTEGRATORS

Content API: AI integration guide

Connect any AI tool so it learns the website's structure by itself, then reads, checks and applies content changes safely.

The basics

ITEM	VALUE
Base URL	`https://backapi.topcasinosg.com.sg/api/v1`
Authentication	`Authorization: Bearer tcs_…` (or `X-API-Key: tcs_…`). Keys are made in the admin dashboard: **Settings** → **API Keys**, by an Admin or Super Admin, and shown once.
Format	JSON in and out. Success: `{"success": true, "data": {…}}`. Failure: `{"success": false, "error": {"message": "…"}}`
Rate limit	120 requests a minute per key. Over it: **429** with a `Retry-After` header.

Three ways to connect an AI tool

① MCP connector

Claude Desktop, Claude Code, any MCP client

Ready-made tools (`get_page`, `save_page_sections`, `get_menus`, `save_theme`…). Set-up in `mcp/README.md`. Easiest for Claude.

② OpenAPI import

ChatGPT GPT Actions, n8n, Make, Zapier, LangChain, any OpenAPI-aware agent

Import `content-api-openapi.yaml` (OpenAPI 3.1, sent with this guide) and set Bearer auth with the key. Every endpoint becomes a tool.

③ Plain HTTP

Your own scripts or agents

Call the endpoints directly. Give the model the system prompt on page 7 and let it call `/me` and `/schema` first.

Two kinds of key

MODE	WHAT IT DOES
Needs approval `review`	Areas: pages, news, media only. Page, news and Global Section changes become **proposals** that an admin approves in **AI Changes**; the response carries a `previewUrl`. Media uploads apply at once. Everything else returns 403.
Direct `direct`	Any ticked area. Changes go **live at once** (new pages and articles start as drafts). Also reaches casinos, bonuses, payment methods, RTP, guides, blacklist, regions, menus, snippets and settings. Deletes go to a 30-day Recycle Bin.

HOW THE AI LEARNS THE SITE

Self-discovery: two calls, no hard-coding

The API describes itself from the live site, so the AI never needs a copy of the structure that can go out of date.

`GET /me`

Who this key is and what it may do:

```
{ "name": "Content AI", "mode": "direct",
  "scopes": ["pages", "news", "navigation"],
  "actingFor": "Jane Tan",
  "requestsPerMinute": 120,
  "whatThisKeyCanDo": [
    "This is a Direct key. Changes go live
     straight away, with no approval step.", … ]
}
```

`GET /schema`

Everything needed to build or edit content:

- **rules**: how a page is built, in plain sentences.
- **endpoints**: only the ones *this key* can use.
- **fieldTypes**: text, richtext, image, button.
- **blockTypes**: every block, its `kind`, what it is for, and its **fields**, with allowed values, which repeat per item, how many sections use them, and a **real example value** from the site.
- **styleFields**, **styleClasses**: spacing, colours, fonts.

The loop that avoids errors

1. **Discover**: `GET /me` and `GET /schema` once per session.
2. **Find**: `GET /pages` (id, path, status), `GET /news`, `GET /global-sections`, `GET /{area}`.
3. **Read right before writing**: `GET /pages/{id}` returns the full ordered `sections` list. Read it again just before each change, so you never overwrite someone else's newer edit.
4. **Edit in place**: change only the field values you need. Keep every other section and field exactly as returned.
5. **Check**: send the change with `"dryRun": true`. Nothing is saved; the answer is `{valid, message, warnings}` or a 400 naming the exact problem.
6. **Apply**: send the same body without `dryRun`.
7. **Verify**: read it back, and tell the user what changed, with ids. Direct changes reach the public site within about a minute.

Why this stays error-free when the site changes

New block types, fields or layouts added by developers appear in `/schema` straight away, with examples taken from pages that already use them. An AI that reads `/schema` first adapts by itself; there is nothing to update on the AI side.

A key acts as the admin who made it, and every change is in the Activity Log as "(API key: name)". It never reaches users, passwords, API keys, security settings or deployments.

CONTENT MODEL

How a page is built

A page is an ordered list of sections: list order is page order. Containers arrange components; depth says what sits inside what.

PART	RULE
Depth 0	Only a `container` or a `global_section` (a shared block, by its `globalId` field).
Depth N	Belongs to the nearest container *before it* at depth N-1. Depth goes up to 4; a container can be at most depth 3.
Container layout	`layout`: `stack`, `row`, `grid`, `tabs`, `accordion`, `carousel`. Tab/accordion children take a `panelTitle` field.
Components	Every other `blockType` from `/schema`. Prefer `kind: "widget"`; `"data"` blocks list the site's casinos, bonuses or articles; `"classic"` blocks are older layouts kept for existing pages.
Fields	`{itemIndex, fieldKey, fieldType, textValue, urlValue, mediaId}`. `itemIndex` 0 = the section's own fields; 1, 2, 3… = repeated items (FAQ questions, slides). Fields marked `perItem` in `/schema` repeat per item.
Values	text and richtext (HTML, scripts removed) in `textValue`; a button: label in `textValue`, link in `urlValue`; an image: `mediaId`.
Style	The `styleFields` (padding, margin, background, font, border, colours) on itemIndex 0, or `styleClasses` presets in `customClass`. Tablet and mobile values: add `Tablet` or `Mobile` to the key (`stPaddingMobile`).

Example: a container with a heading and a two-line icon list

```
"sections": [
  { "blockType": "container", "depth": 0, "fields": [
      { "itemIndex": 0, "fieldKey": "layout", "fieldType": "text", "textValue": "stack" } ] },
  { "blockType": "heading", "depth": 1, "fields": [
      { "itemIndex": 0, "fieldKey": "heading", "fieldType": "text", "textValue": "Why play here" } ] },
  { "blockType": "icon_list", "depth": 1, "fields": [
      { "itemIndex": 0, "fieldKey": "color", "fieldType": "text", "textValue": "success" },
      { "itemIndex": 1, "fieldKey": "icon", "fieldType": "text", "textValue": "bolt" },
      { "itemIndex": 1, "fieldKey": "text", "fieldType": "text", "textValue": "Fast withdrawals" },
      { "itemIndex": 2, "fieldKey": "icon", "fieldType": "text", "textValue": "shield-check" },
      { "itemIndex": 2, "fieldKey": "text", "fieldType": "text", "textValue": "Licensed and audited" } ] }
]
```

List settings on itemIndex 0; each line is an item (1, 2…). `/schema` lists the allowed values (colours, icons).

Editing rule

Send the **whole** `sections` list back, edited: a section left out is removed from the page. Global Sections work the same way; read `usedOn` first, as every listed page changes.

REFERENCE

Endpoints

ENDPOINT	WHAT IT DOES
`GET /me` · `GET /schema`	This key; the content model and this key's endpoints.
`GET /pages` · `GET /pages/{id}`	Pages (id, parentId, path, title, SEO, status); one page with `sections`.
`POST /pages`	New page `{title, slug, parentId, metaTitle, metaDescription, sections, note, dryRun}`. Direct: created as a draft.
`POST /pages/{id}/changes`	Change `{title, metaTitle, metaDescription, sections, note, dryRun}`; anything left out stays. Slug and parent are not changed here.
`GET /global-sections` · `GET /global-sections/{id}`	Shared blocks; one with `sections` and `usedOn` (the pages that show it).
`POST /global-sections/{id}/changes`	`{name, sections, note, dryRun}`. Admin's key only. The dry run warns which pages change. No create or delete.
`GET /news?page=&pageSize=` · `GET /news/{id}`	Articles; one with its HTML `content`.
`POST /news` · `POST /news/{id}/changes`	`{title, slug, excerpt, content, coverMediaId, note, dryRun}`.
`GET /media` · `POST /media`	Media Library; upload as multipart field `file`, then use its `id` as `mediaId`.
`GET /changes?status=` · `GET /changes/{id}` · `POST /changes/{id}/withdraw`	Changes this key sent: pending, approved, rejected (with the reviewer's note) or withdrawn.

What a change returns

```
// dryRun: nothing saved
{ "valid": true, "message": "This change is valid. Send it again without dryRun to apply it…",
  "warnings": ["This Global Section is on 3 page(s), which all show the change: /sg, /my, /th"] }

// Direct key: applied (201)
{ "change": { "id": 88, "contentType": "page", "action": "update", "status": "approved",
              "targetId": 12, "resultId": 12, … },
  "message": "Done. The change is live.", "warnings": [] }

// Needs-approval key: waiting (201)
{ "change": { "id": 89, "status": "pending", … }, "previewUrl": "https://…",
  "message": "Submitted. It goes live only after an admin approves it…", "warnings": [] }
```

REFERENCE

Direct keys: every other area

PATTERN	WHAT IT DOES
`GET /{area}` · `GET /{area}/{id}`	List; read one with all its fields.
`POST /{area}` · `PUT /{area}/{id}`	Create; update. Send **all** fields as GET returned them, with your edits.
`PUT /{area}/{id}/status`	`{"status": "draft" | "published" | "scheduled", "publishAt": RFC 3339 | null}`. Also for pages and news.
`DELETE /{area}/{id}`	To the Recycle Bin (pages, news, casinos, bonuses, payment-methods, rtp, guides, blacklist, snippets). Other types: 403.
`GET /recycle-bin` · `POST /recycle-bin/{id}/restore`	What was deleted (30 days); undo, with the previous status.
`GET/PUT /seo/{page|news|casino|bonus|guide}/{id}`	SEO fields.

AREA (SCOPE)	PATHS
casinos	`casinos`, `game-providers`, `licenses`
bonuses · guides	`bonuses` · `guides`
payment_methods · rtp · blacklist	`payment-methods` · `rtp` · `blacklist`
regions	`regions`; `PUT /regions/{id}/active {"active": bool}` instead of delete
navigation	`menu-items?location=header|footer` `{location, parentId, label, href, sourceType: static|dynamic_regions|dynamic_casinos, sortOrder}`; DELETE removes for good, only with nothing under it (409). `sidebar`: GET, PUT whole.
snippets	`snippets`, `PUT /snippets/{id}/active`, `pages/{id}/snippets`
settings	`settings/site` (GET includes footer; PUT), `PUT settings/footer`, `settings/theme`: GET lists colour tokens with default and current value; PUT `{"colors": {"primary-500": "#3e4ba8"}}` **replaces the whole theme**. Menus, theme and Global Sections need the key's admin to be an Admin or Super Admin.

GETTING IT RIGHT FIRST TIME

Built-in safeguards and errors

What the API checks before saving anything

- **Unknown JSON fields are refused**, by name, so a typo can't silently drop a change.
- **Block types, depths and nesting** are checked against the rules in `/schema`; **field types** must be text, richtext, image or button.
- **Every mediaId, globalId, parentId and coverMediaId must exist**; slugs must be lowercase-with-hyphens and free.
- **Warnings** (not errors): a field no section of that type uses, so it may not show; and for Global Sections, which pages change. Read `warnings` on every response.

Typical messages and the fix

MESSAGE (400 UNLESS NOTED)	FIX
`sections[3]: unknown blockType "hero2" (GET /api/v1/schema lists them)`	Use a `blockType` from `/schema`.
`sections[2]: "icon_list" is a component, so it must be inside a Container…`	Put a container before it and give it depth 1.
`sections[4]: depth 2 needs a Container at depth 1 before it…`	Fix the depths: each level needs its container.
`sections[1].fields[0]: mediaId 99 is not in the Media Library…`	`GET /media` or upload with `POST /media`.
`the request body is not valid JSON for this endpoint: unknown field "heading"`	Send only the documented fields.
`an update can't change the page's slug or parent…`	Direct key: `PUT /pages/{id}`; otherwise ask an admin.
`a page already exists at /…` · `another article already uses the slug…`	Pick another slug (one in the Recycle Bin still counts).
`menu item 12 has 2 item(s) under it…` (409)	Move or delete the child items first.

STATUS	MEANING
400	Invalid request; the message names the field.
401	Missing, wrong or revoked key, or its admin was disabled.
403	Wrong mode, area not ticked, not deletable, or Admin-only (Global Sections, menus, theme).
404	No such item, or a path keys can't use.
409	Conflict: slug taken, already in the bin, items under a menu item, or a proposal outdated by a newer edit.
429 · 5xx	Wait (`Retry-After`) and retry. Before retrying a **create**, list first: the first call may have worked.

COPY INTO THE AI TOOL

Instructions for the AI

Paste this into the tool's system prompt or instructions (ChatGPT: the GPT's Instructions; Claude: Project instructions; n8n/Make: the agent's system message).

```
You manage the TopCasinoSG website through its Content API
(https://backapi.topcasinosg.com.sg/api/v1, key sent as "Authorization: Bearer").

Learn first
1. At the start, call GET /me (your mode and areas) and GET /schema (rules, every
   blockType with its fields, allowed values and real examples, and your endpoints).
   Never guess a blockType, fieldKey or endpoint: use only what /schema lists.

Change safely
2. Find the item (GET /pages, /news, /global-sections, /{area}), then read it
   (GET .../{id}) right before changing it.
3. Change only what the user asked. Pages and Global Sections: send the WHOLE
   sections list back, edited, keeping every other section and field exactly as
   returned. PUT /{area}/{id}: send all fields as GET returned them.
4. Always send the change with "dryRun": true first. If it fails, read the message
   (it names the field), fix that, and dry-run again. Then send it without dryRun.
5. Read every "warnings" list and tell the user about them.
6. Images: use a mediaId from GET /media, or upload with POST /media first.

Ask before
7. Publishing, deleting, changing many items, or anything site-wide (Global
   Sections: check usedOn; menus; the theme, where PUT replaces the whole theme,
   so start from GET /settings/theme). Say exactly what you will do; wait for yes.
8. New pages and articles start as drafts; publish only when asked
   (PUT /{type}/{id}/status {"status": "published"}).

Afterwards
9. Read the item back and report what you changed, with ids, in plain words.
10. On 429 or 5xx wait and retry; before retrying a create, check it wasn't made.
11. Undo a delete: GET /recycle-bin, then POST /recycle-bin/{id}/restore.

Writing: match the site's tone and spelling; never invent bonus amounts, licences,
RTP figures or other facts; no scripts or styles in HTML.
```

Testing the connection

Ask the AI: *"What can you do with this key?"* It should call `/me` and answer with the mode and areas. Then try a dry run: *"Check, without saving, a change to the About Us page title."*

Keep the key secret

Put it in the tool's authentication settings, never in a chat message or a prompt. One key per person or tool. If it leaks: **Settings** → **API Keys** → **Revoke**.